

Evidence Crosses, Authority Does Not: Receiver-Owned Admission of Agent Tool Calls Within and Across Organizations

The Chio Project

<https://github.com/bb-connor/chio>

September 2026

Abstract

Agent tool calls need authorization that survives retries, delegation, and changes in the receiver’s state. Chio places the effective admission decision inside the kernel that dispatches, under roots the receiver activated before the request. Requests carry signed evidence and references to receiver-held state; they cannot install trust roots. The kernel binds the request to an agreement and a single-use continuation, checks locally activated lease and governance records, and signs durable decision evidence. An inbound co-signed statement gates admission; an outbound statement binds the resulting receipt and gates response delivery. Its countersigner reconstructs the typed signing preimage and verifies the participant bindings. We enumerate the implemented comparisons, mechanize their classification, and exercise substitutions through actual tool dispatch. A standards-based comparison identifies missing semantics in our selected deployment profile, while a receiver-issued-token counterexample shows that receiver ownership is also available within those standards. The candidate contribution is a reusable execution and evidence contract across organizations, not a new cryptographic primitive or an impossibility result about existing token formats.

1 The rule

A buyer agent at one company asks a vendor at a second to stage a refund. The vendor must decide before touching its ledger whether these arguments, under this agreement, remain authorized and whether the permitted effect was already attempted. Caller identity alone does not answer those questions. Retries and replacement agents must not manufacture another authorization.

Effective authority is decided inside the kernel that dispatches, against roots the receiver activated itself; nothing carried in a request can add a root. Signed, content-addressed evidence crosses the organization boundary. The kernel may trust configured capability issuers, but a credential has effect only through this local decision. The receiver owns its active agreements, pinned keys, validity windows, budgets, and continuation-consumption state. This applies established end-to-end and trust-management principles [20, 6, 12] to the dispatch boundary.

A kernel-signed receipt records the decision. An inbound co-signed statement names receiver-held evidence and gates admission. An outbound statement binds the newly durable receipt and gates the response; its countersigner reconstructs the typed preimage and checks participant identities and key fingerprints. That check does not independently reproduce the receiver’s policy evaluation. Authorization, outcome, and accounting records share cryptographic bindings; the property does not require them to be one physical object.

The interface specifies which evidence is authenticated, which local state it resolves, and when that state is consumed. Some facts travel as explicit references; others can be recovered through a receiver-issued opaque handle. OAuth permits that design [11, 15]. Our comparison measures one selected profile and its omitted bindings, not the expressive limits of those formats. The research question is whether independent organizations can procure, delegate, verify and settle agent work through a common contract while retaining local authority. Discovery, messaging and market evidence must bind to that contract. Our experiments establish local implementation properties; independent deployment and reduced application-specific recovery remain open. Python–Rust work, bounded subcontracts and agreed releases establish neither independent operation nor novelty.

2 Threat model and trusted base

The kernel sits between an agent and the tool servers whose transport it owns: child processes, fronted HTTP tools, proxied sidecars. It does not confine the agent, whose model, orchestrator, prompt, and libraries are untrusted; one that opens its own socket reaches whatever that socket reaches.

The adversary is given everything outside the receiving kernel: every field this paper names, the peer kernel and its signing key, the network, where it may drop, reorder, and replay, and any receipt, statement, lease, or governance record it has observed. It holds neither the receiving kernel’s signing key nor write access to its stores. The question is what it can cause a receiver to dispatch.

The trusted base is the receiving kernel’s process, its signing key, and its stores; everything outside is verified or assumed; the assumptions appear once, in Table 1.

Table 1: Assumptions outside the receiving kernel, by the identifiers of the implementation’s assumption registry. The registry has 16 entries; the ten this protocol rests on are listed.

Identifier	What is assumed
ASSUME-ED25519	Ed25519 meets existential unforgeability under chosen-message attack for keys the kernel trusts.
ASSUME-SHA256	SHA-256 is collision resistant over receipt and statement preimages.
ASSUME-CANONICAL-JSON	The production canonical encoder agrees byte for byte with the mechanized one on strings, bounded integers, finite arrays, and objects ordered by UTF-16 code unit. Floats are outside that domain.
ASSUME-SQLITE-ATOMICITY	A single-row insert commits atomically, so a primary-key conflict is a reliable witness that a row existed. Cross-row recovery and ordering are not assumed.
ASSUME-OS-CLOCK	The injected clock reports Unix time within the operator’s declared tolerance.
ASSUME-NETWORK-TRANSPORT, ASSUME-TLS	Delivery is not assumed reliable; arriving messages are assumed not rewritten below the signature checks, and remote surfaces authenticate their endpoints.
ASSUME-SUBPROCESS-ISOLATION	Effects after an allow, and isolation between tool processes, rest on operating-system boundaries.
ASSUME-FEDERATED-ORIGIN-CLASSIFICATION	The receiving edge marks cross-organization origin from a transport-authenticated peer identity; a request misclassified as local never reaches these checks.
ASSUME-GOSSIP-FAIRNESS-PARTITION-BOUND	Revocation liveness only: recurring delivery under weak fairness, bounded skew, operator-bounded partition healing. Safety does not depend on it.

Two assumptions are deliberately absent. The first is that the two signing keys on a co-signed statement belong to different organizations: one operator holding both satisfies every check here. The second is domain separation: one passport key signs three preimage families, the receipt signing body, the co-signing body, and a statement’s pre-authentication encoding, so ASSUME-ED25519 is an assumption about a key with that reuse. A pinned peer obtains this kernel’s signature over any bytes that rebuild as one of the last two, so a countersignature fixes a record’s shape and parties, not its truth.

3 The receipt

A receipt is a JSON object with a fixed field set, encoded canonically under RFC 8785 [19]. Three of its projections are hashed, and they nest: b drops the `id`, the `signature`, and the two optional fields recording how it was signed; the canonical receipt body β is b plus the `id`; r is the whole receipt as it travels. Write D for SHA-256 of the canonical encoding and k for the kernel key published in the body; signatures are Ed25519 [16].

$$id = D(b), \quad \sigma = \text{Ed25519}_k(\text{JCS}(\{\text{body}: b, \text{id}: id\}))$$

Each projection has its own wire field: id names the receipt, $D(\beta)$ is the subject digest of a co-signed statement, inbound or outbound, and $D(r)$ is a binding reference’s `remote_receipt_sha256`. No two agree, because β adds the `id` to b and r adds σ to β . The signature covers a fourth preimage, the object above, so that field is covered by σ and not by id . A receipt renamed is a receipt unsigned. Three properties follow.

Recompute and refuse. Where the path holds the evaluated content the signer is handed it, not a digest: it recomputes the digest inside the trust boundary and refuses to sign if the body’s claimed digest differs.

Decisions, not successes. A denial is a receipt of the same shape, built, signed, and persisted on the same path as an allow. Three fields fix what kind of statement a receipt is, so a weaker one cannot pose as a stronger one: `receipt_kind` separates a mediated decision from a passive trace, `boundary_class` says whether

the kernel could have prevented the effect or only observed it, and `trust_level` records the evidence class. A body claiming a decision on a kind that cannot make one is not signed, and a message rejected before it had a decision returns its code and no receipt.

Permission, proof, and price in one object. When the exercised grant carries monetary caps the hold taken before dispatch is reconciled into the receipt’s financial block, so the object an auditor reads to learn the call was allowed is the one a biller reads to learn what it cost. Settlement between organizations is out of scope, and that is our whole claim about commerce.

The vendor kernel’s receipt for the refund step, abridged at the metadata:

```
{ "id": "d342a59d...6fba2e3f", "timestamp": 1766000001,
  "capability_id": "lease-vendor-c-refund", "tool_server": "vendor-c.payments",
  "tool_name": "stage_refund",
  "action": { "parameters": { "caseRef": "refund-250", "tool": "stage_refund",
    "workflowId": "wf-chio-refund-001" },
    "parameter_hash": "47e6e096...d5a6464b" },
  "decision": { "verdict": "allow" }, "trust_level": "mediated",
  "receipt_kind": "mediated_decision", "boundary_class": "prevent",
  "tool_origin": "caller_executed", "redaction_mode": "none",
  "content_hash": "c9abf68a...d9803987",
  "policy_hash": "82cfea63...fb2ca3fd",
  "metadata": { "attribution": { "delegation_depth": 0, "grant_index": 0, ... },
    "chio_runtime": { "accepted": true, "destructive": true,
      "lease_id": "lease-vendor-c-refund", "step_index": 2,
      "reserved_treaty_continuation_id": "continuation:runtime-loopback:2" }, ... },
  "kernel_key": "31debe55...cfa498bc",
  "signature": "a9e4762e...1ba28505" }
```

Checked on these bytes: b hashes to `d342a59d...6fba2e3f`, the object’s id; β to `31118bb6...4781b3f9`, the subject digest of the statement the vendor mints over it after persisting it (Section 5); r to `63f1f35a...8c93e3e8`, that statement’s `remote_receipt_sha256`. The signing preimage over b verifies under the embedded kernel key.

4 Admission

Admission is the one operation. Every tool call, local or foreign, goes through it, and every path that reached a decision produces a signed receipt.

1. Verify the presented capability against the kernel’s own trusted issuer set: signature, time window, subject binding, and the revocation state of the leaf and every ancestor; resolve the grants matching this tool and these arguments.
2. Open a durable admission operation, a fenced write-ahead record whose state machine has 18 states. Its fence is a store lease with an epoch: no second coordinator can advance it.
3. Run the guard pipeline. Deny dominates, and so does error: a guard that produces no verdict denies.
4. Run the admission hook, where Section 5 runs for a request classified as foreign; one carrying no agreement context is denied here.
5. Take the budget hold, charging the reservation before dispatch.
6. Revalidate the mutable inputs, mark the operation dispatch-committed, and only then hand the call to the tool server over a transport the kernel owns.
7. Run output guards, build the receipt, recheck its fields against the admitted inputs, sign and persist it, then return.

Three orderings are the whole of its value. **Durable before dispatch:** no dispatch occurs without a committed pre-dispatch record. Before that commit recovery releases the hold; after it, with no outcome known, recovery terminates the operation as outcome-unknown rather than inventing one. An output not matching the committed digest terminates as denied-after-delivery, with a signed deny and nothing captured against the hold.

Receipt before response: the receipt is persisted before the caller sees the result, and a kernel that cannot persist a receipt denies. The log is a write-ahead record of what was admitted, not a trace of what was observed.

Consume after compare: step 4 spends a foreign request’s single-use continuation after every comparison against the inbound statement it presented and both its signatures verify, and before the local admission report that rebuilds this call’s binding from the live request. A statement the receiver refuses cannot burn a continuation a legitimate call still needs; one that passes every comparison on a request the report then denies does spend one and gets it back through a release scoped to that admission, recorded as ambiguous if unconfirmed.

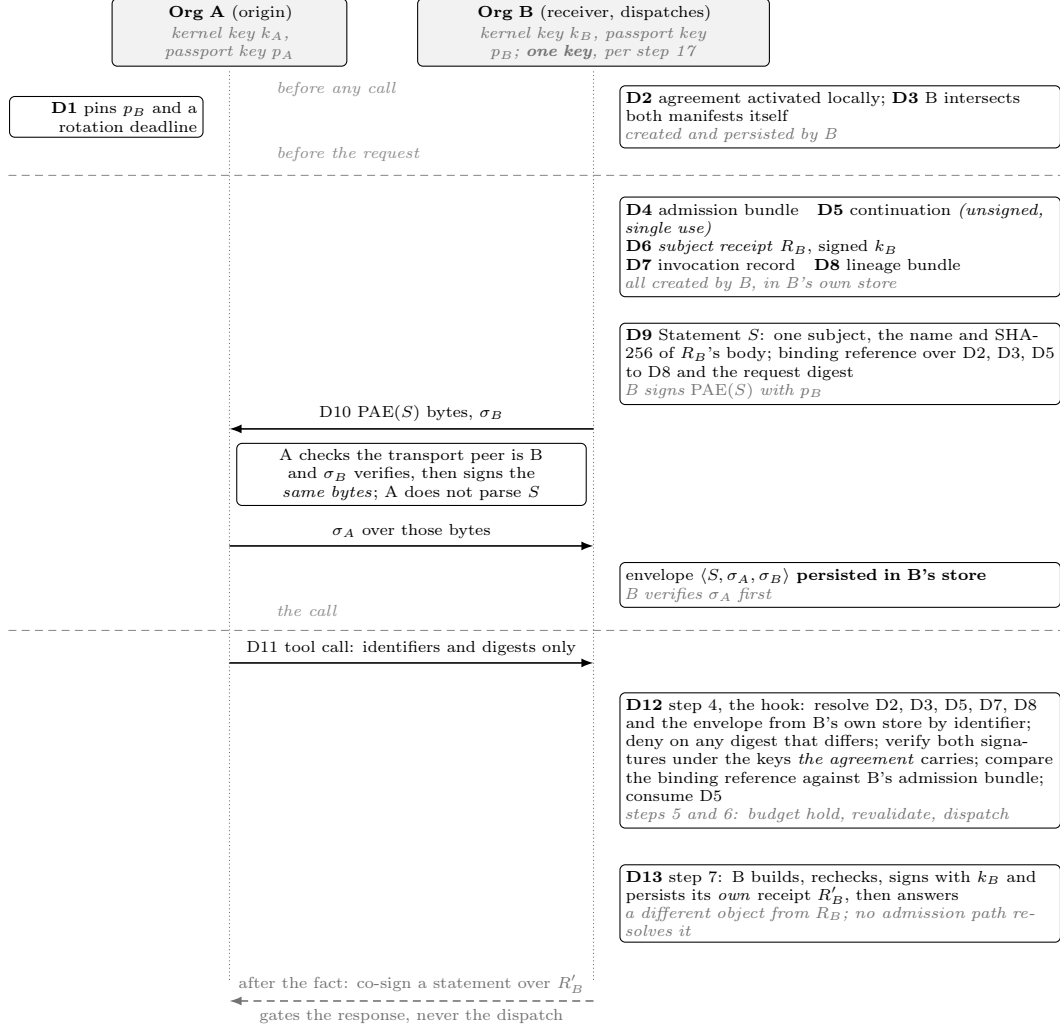


Figure 1: One cross-organization call, end to end. Every object the receiving kernel checks at step 4 was created by the receiver and was already in the receiver's own store before the request arrived; the request carries identifiers and digests and nothing else. The statement the hook verifies (D9, D10) has as its subject a receipt the *receiver* signed (D6), not the peer's, and that is a different receipt from the one the receiver writes for this call at step 7 (D13). The dashed return is the outbound co-signature: the reference kernel mints it on every federated request that produced verified treaty material and fails the response if it cannot, but it is minted after the dispatch it describes and no admission path resolves it.

5 Cross-organization admission

A call from another organization is the same admission operation, and both co-signed statements occur in it. The request names the inbound one and the pre-dispatch hook resolves it at step 4, adding no authority: the receiver's issuer set, pinned peers, agreements, and clocks are what they were before the request arrived. Section 6 concerns it, and Figure 1 follows one call end to end.

5.1 Receiver-owned resolution

Two organizations pin each other out of band, each recording the other's kernel identity, the passport key it co-signs with, and a rotation deadline. The agreement is an activated record naming its participant kernels, the action classes in scope, a validity window, and a revocation epoch, of which exactly two are enforced where co-signed evidence is checked. Each participant publishes a signed manifest, the action classes it accepts and at what intensity on the ascending ladder **observation**, **guarded**, **receipt_backed**, **partition_contingency**, **maintenance**, and each receiver activates the agreement and both manifests locally and intersects them per

action class: the higher-intensity mode and the stricter co-signing requirement win, and a destructive class below `receipt_backed` rejects. A pin the receiver has not activated is a denial.

The request may carry identifiers. It may not carry trust. Its agreement context is names and digests, each a lookup key into a store the receiver owns; Figure 2 is the whole of what such a name reaches. Refused wherever they appear in that context, at any depth, in objects and inside arrays, are 7 key names for a trust root, bundle, agreement record, manifest, signing key or peer directory, with `request_smuggled_trust_root`, and 4 for dynamic trust input or peer discovery, with `request_smuggled_dynamic_trust`. The traversal is bounded at 32 levels and 4096 entries, and a context past either bound is refused rather than half scanned, so the refusal is total and the work one request can demand is capped.

5.2 The co-signed statement

One wire form serves both directions: an in-toto Statement v1 [23] in a DSSE envelope [21] whose predicate is `snake_case` and rejects unknown fields. Its single subject names an invocation identifier and carries the SHA-256 of that receipt’s canonical body of Section 3; the vendor’s outbound refund envelope and its payload follow. An inbound subject instead names a receipt the receiver already held, in the reference path one record per process on every call, so the per-call binding rests on the binding reference alone.

```
{ "payloadType": "application/vnd.in-toto+json",  
  "payload": "eyJJdHlwZS16Imh0dHs2b284VnV4dG90b3Y5b3Yz",  
  "signatures": [  
    { "keyid": "fdf72a088f18d7399e85c2bce4484150f1759a595a86980e5d9a422a01e6d55",  
      "sig": "3Q029J3P4YfLnvqzy5PwYfUz0X0U/DBCT7yFB8pbA6iBguyRt1+iz17A...",  
      "keyid": "3c7abdc79fd08dbd495a803952f29ef8a78b4684b63591631cfef8e8871c8",  
      "sig": "R0i0t8Zc2EPARqU4mcbU0uVH0z8p59m8m3XcT0DxdWf48btYzmd..." } ] }
```

```
{
  "in_type": "https://in-toto.io/Statement/v1",
  "predicateType": "chlo.bilateral-co-sign-involution.v1",
  "subject": [ {
    "name": "chlo-receipt:d342a59d...f6ba2e3f",
    "digest": { "sha256": "31118bbe0...478133f9" } },
    {
      "name": "stage-refund",
      "digest": { "sha256": "17660000001000..." } } ],
  "predicate": {
    "invocation_id": "d342a59d...f6ba2e3f", "tool_name": "stage_refund",
    "tool_server_a": { "kernel_id": "did:chlo:buyer-kernel", "alg": "ed25519",
      "passport_key_fingerprint": "ffdf72a08...a01e5d55",
      "scope_digest": { "alg": "sha256", "value": "476e069e...d5a6464b" } },
    "tool_server_b": { "kernel_id": "did:chlo:vendor-c", "alg": "ed25519",
      "passport_key_fingerprint": "e37abc47...8ee871c8",
      "scope_digest": { "alg": "sha256", "value": "262768fe...b9f835aa" } },
    "co_sign": "bilateral_required", "consistency_model": "totally-ordered",
    "consistency_anchor": "chlo:consistency-vf-chio-refund-001:2",
    "cross_org_visibility": "federated", "timestamp_unix_ms": 17660000001000,
    "tool_args_hash": { "alg": "sha256", "value": "476e069e...d5a6464b" },
    "capability_lease_refs": [ { "lease_id": "lease-vendor-c-refund",
      "issuer": "did:chlo:buyer-kernel", "expires_at_unix_ms": 1766000030000,
      "scope_digest": { "alg": "sha256", "value": "262768fe...b9f835aa" } } ],
    "governance_receipt_refs": [ { "receipt_id": "gov-refund-stage-authorization",
      "kernel_id": "did:chlo:buyer-governance",
      "digest": { "alg": "sha256", "value": "21143a93...f161e3c0" } },
      { "kernel_id": "did:chlo:vender-c",
        "digest": { "alg": "sha256", "value": "476e069e...d5a6464b" } } ],
    "policy_evaluation_summary": { "joint_disposition": "allow",
      "server_a_verdict": { "verdict": "allow", "policy_id": "buyer-policy:stage_refund", ... },
      "server_b_verdict": { "verdict": "allow", ... } },
    "treaty_binding_ref": {
      "treaty_id": "treaty:runtime-loopback:2",
      "action_class_id": "workflow_cross_kernel_stage_refund",
      "consistency_model": "totally-ordered",
      "treaty_scope_sha256": "0bf230a9...dif2b21c",
      "ladder_interaction_sha256": "93dc7cf...af7943d1",
      "admission_report_sha256": "91d74509...f1645221",
      "continuation_sha256": "86cf2348...366b93f4",
      "lineage_bundle_sha256": "28d816a3...d3b504ba",
      "request_sha256": "47e6e096...d5a6464b",
      "outcome_sha256": "c9abf68a...d9803987",
      "local_receipt_sha256": "9cbe4438...ad017121",
      "remote_receipt_sha256": "63f1f35a...8c93ac3e8",
      "lease_refs": [ "lease-vendor-c-refund" ],
      "governance_refs": [ "gov-refund-stage-authorization" ],
      "signer_kernel_ids": [ "did:chlo:buyer-kernel", "did:chlo:vender-c" ] ] } }
```

Both signatures cover the DSSE pre-authentication encoding of the payload, and each `keyid` is the SHA-256 of its raw public key. Recomputed from the committed bytes, `request_sha256` is `tool_args_hash.value` and the receipt's `action.parameter_hash`.

5.3 The verifier, in the order it runs

Table 2 is the shipped order; two components decide, in two disjoint code families. The conforming verifier returns the specification’s dotted codes, a closed set of 24 strings of which it emits 16 and the envelope layer the rest. The kernel’s pre-dispatch hook answers from its runtime failure-code set of 526 strings, 24 of them agreement-scoped, and maps every envelope-layer failure to `chio_treaty_unverified_required_evidence`. Table 3 and Section 8’s denial experiment report the hook, which decides live calls.

Signatures are verified last and the key lookup first, the opposite of the usual advice: neither decider can verify until it knows which keys to verify under, neither takes them from the envelope, and no gate before

Table 2: The conforming verifier, 26 steps, grouped by where the state each step reads comes from. Every failure returns one code from the closed set; the specification maps each step to its own. Steps 8 to 14 are the envelope layer, called with the two keys the receiver pinned.

#	Check
1–5	Media type, at least one signature, payload decodes and parses; predicate type chio.bilateral-cosign-invocation.v1 ; Statement v1; exactly one subject
6–7	Both kernel identifiers are in the pinned peer set and each pin’s fingerprint equals the declared one
8–13	Exactly two signatures; re-canonizing reproduces the payload bytes; predicate schema and strict rules; both verdicts present, equal, and in {allow, deny}; subject name is chio-receipt: plus the invocation identifier; the two pinned keys and their identifiers distinct
14	<i>Only now</i> : compute the pre-authentication encoding, select each signature by a pinned key’s identifier, and verify both
15–16	Each pinned peer’s manifest reference is present and fresh, and the peer is active at the pinned epoch height
17–19	Resolve the receipt by invocation identifier in the receiver’s own store; its signature verifies, its tool name and argument hash agree, its kernel key is the pinned second-party key, and subject name and digest match the resolved body
20–22	Both verdicts resolve and agree; the lease resolves with matching issuer, expiry and scope digest and is unexpired; the tool name is in the action-class table; a receipt-backed class resolves its governance record
23–26	Binding cross-checks (Table 3), consistency model and anchor, co-signing mode; return the verified material

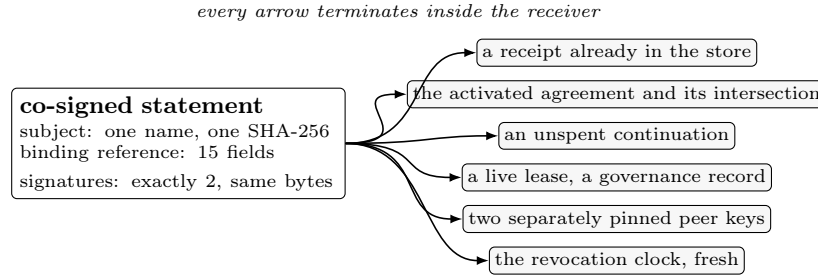


Figure 2: The mechanism. The statement carries no authority of its own: its subject, the binding fields the receiver compares, and each of its two keys resolve only against records the receiving kernel placed in its own stores before the request arrived.

step 14 consults a secret, so the ordering costs nothing in soundness. The verifier reads those keys from its pin set and the hook from the activated agreement, so a rotation carried into one and not the other is admitted by whichever decider still holds the key the envelope names. Steps 17 to 19 are the verifier’s alone: on a live call neither decider resolves a receipt by invocation identifier or compares the subject digest, so the receipt binding rests on `local_receipt_sha256` and `remote_receipt_sha256` against the lineage bundle and the invocation record.

5.4 Consume, bind, co-sign

A replayed statement is validly signed, so no signature defeats replay. A single-use continuation does, and the receiver mints it before the call: a record binding the parent receipt and session anchor, the capability, the action class, the target tool, a nonce, the two kernel identities, and a validity window, whose digest is one of the 15 bound fields. It carries no signature and needs none: residency in the receiver’s own store and the digest the request presents authenticate it. Two operations run on one table keyed by that continuation identifier. *Consume* inserts, ignoring a conflict, and reads the affected row count; zero rows means it was already spent and the call is denied with `chio_treaty_continuation_replay`. *Release* deletes the row for that identifier and that admission, and runs only on a later pre-dispatch denial. Once dispatch commits the row stays: at-most-once, on purpose.

The receiver then builds its own admission report and binds the accepted material into the receipt it signs, checking the statement’s `admission_report_sha256` only for well-formed hex before overwriting it with its *own* report’s digest.

The outbound statement is minted here, a different object: its subject is the receipt this call just produced, whose pre-authentication bytes the receiver authors and the peer signs without parsing. The local receipt is

persisted first, so a refusal or transport failure is recorded against an existing receipt. The reference kernel mints it whenever a request produced verified treaty material, fails the response when it cannot, and mints none for a request denied before that. No admission path resolves it. Its second signature constrains nothing the receiver decided, since the Section 2 adversary holds the peer’s key: evidence, not prevention.

6 Security

Four properties carry the protocol, each over the adversary of Section 2 and reduced to an assumption of Table 1. All four concern the inbound statement; the outbound one gates the response, never the dispatch.

Admission binding. *If a receiving kernel dispatches a federated tool call, then both keys the activated agreement pins signed the presented statement S ’s pre-authentication bytes; every digest the request presented equalled that of the receiver-stored artifact its identifier resolved to, and S ’s binding reference names that digest for the agreement, the intersection, the continuation and any lineage bundle; every other compared leaf of S equals what the receiver computed for this request, save two bound to a receiver-held set, the lease expiry to the presentation window below and the issuer to the agreement’s participants; and the continuation S names was unconsumed at admission.* Suppose not. Each equality is byte equality between a digest the receiver computed and one the statement carries, so a difference that passed means two objects share a SHA-256 image or a canonical encoding, contradicting ASSUME-SHA256 or the encoder’s injectivity under ASSUME-CANONICAL-JSON; a differing signature contradicts ASSUME-ED25519, a twice-consumed continuation ASSUME-SQLITE-ATOMICITY. Two residuals remain, both taken up in Section 7: completeness by exhaustiveness over the leaf type, and that the shipped hook realizes the enumerated predicate.

Nothing in the agreement puts the statement in that antecedent: the resolved co-signing mode adds the invocation record to required evidence at both `bilateral_required` and `bilateral_if_cross_org`, but never adds the statement itself. The kernel closes it above the hook, refusing a federated call the hook allowed carrying no verified statement. The three preimages of Section 2 carry these properties under ASSUME-ED25519 only if disjoint, which here is structural: a reserved first byte for the encoding, disjoint key sets for the others.

Receiver locality. *Every root, pin, agreement, action-class table and registry used to decide a request is state the receiver held before the request arrived; the request selects among them and contributes none.* Extraction reads a fixed set of key names and ignores the rest; each is an identifier, the argument of a total lookup over receiver-owned state.

Single use. *For a continuation identifier c , at most one admission ever reaches dispatch with c bound to it.* Under ASSUME-SQLITE-ATOMICITY exactly one of a set of concurrent inserts on a primary key creates the row; the rest deny. Release deletes only the row keyed by c and the admission that took it, and runs only before dispatch (Section 4). And c is resident in the receiver’s store before the request, authenticated by residency and digest rather than signature, so the receiver decides how many authorizations exist. The reference path mints it there; origin minting, also permitted, owes an unpredictability premise.

Audience binding. *A statement accepted by receiver R is not accepted by a different receiver R' unless R' activated an identical agreement, resolved the same intersection, and holds the same continuation unconsumed.* The receiver’s identity enters through the continuation, not the statement’s name for it: the continuation R' resolved names R' as target and both kernels as participants of the agreement R' activated, which has exactly two, so R' admits only as its second, under the key it pins. Another validly co-signed statement from the same pair stops there too, naming a continuation the receiver does not know or has consumed.

Substitution over the binding reference. Table 3 is an executed corpus over the 15 binding fields, evidence PS-T12; binding-substitution denials leave the continuation unconsumed and dispatch no tool. Additional cases change the receiver’s activation records and deny both presentations after store reopen.

Table 3: Substituting one binding field at a time with a well-formed different value, both signatures valid over the substituted statement, as decided by the pre-dispatch hook. The signer list appears twice, as a set and in order, and the four conditional rows are compared only when the action class’s required-evidence set makes the admission resolve the artifact they bind. Each case re-signs with both participant keys, so both signatures verify over the bytes the hook decides on; of the 37 cases one substitutes nothing and admits, so every denial is attributable to the field that changed. This corpus and the leaf corpus below both run every case through a kernel whose one registered tool server counts its own invocations, 566 evaluations over 283 cases, all of both: each case routes through a single driver that drives the substituted request and then replays the unsubstituted one against the same store. The count is two-sided, which is what makes zero mean something: every denial asserts it reads zero and every admission asserts it reads exactly one on the same wiring. **binding_mismatch** and **unverified_evidence** abbreviate **chio_treaty_dsse_binding_mismatch** and **chio_treaty_unverified_required_evidence**.

Field	Receiver-side value it must equal	First code	Compared when
treaty_id	the agreement the receiver resolved for this request	binding_mismatch	always
treaty_scope_sha256	the digest the receiver computes over its own agreement record	binding_mismatch	always
ladder_intersection_sha256	the intersection digest the receiver resolved	binding_mismatch	always
admission_report_sha256	<i>nothing</i> : 64 hex characters, then overwritten by the receiver’s own report digest	none: the substituted call is admitted and dispatches	never
continuation_sha256	the resolved continuation’s digest	binding_mismatch	always
action_class_id	the action class the receiver resolved for this request	binding_mismatch	always
consistency_model	the model the resolved intersection fixed	binding_mismatch	always
request_sha256	this dispatch’s argument hash, and the predicate’s own tool_args_hash	binding_mismatch	always
lease_refs	the lease in the receiver’s own admission bundle	binding_mismatch	always
governance_refs	the governance record in that bundle	binding_mismatch	always
signer_kernel_ids , as a set	exactly two, equal as a set to the agreement’s participants	binding_mismatch	always
signer_kernel_ids , in order	the invocation record’s order; a reversal resolves the two pinned keys transposed, and strict envelope verification refuses that pairing before the ordered comparison runs	unverified_evidence	always
lineage_bundle_sha256	the lineage bundle the receiver resolved	binding_mismatch	lineage required
outcome_sha256	the outcome digest in the receiver’s invocation record	binding_mismatch	record required
local_receipt_sha256	the lineage bundle’s root receipt digest, and the invocation record’s	binding_mismatch	either required
remote_receipt_sha256	the lineage bundle’s leaf receipt digest, and the invocation record’s	binding_mismatch	either required

That table is one sub-object of the comparison set, not the set. A strict statement’s wire type has 55 leaves; the profile refuses 2, leaving 53 a statement may carry and 49 this call carries: 27 compared for equality against a named receiver-held value, 1 for membership in a receiver-held set, 13 against the statement alone, by a declared constant or a second leaf, and 8 against nothing the receiver holds. 245 executed cases fix that classification leaf by leaf.

Those 8 include the report digest, the consistency anchor, the policy identifiers, and the subject digest, which only the conforming verifier of Section 5 checks. Each is still held to its declared shape: a substitution within it moves no decision, while a non-hexadecimal report digest and an empty policy identifier are denied. The 4 optional leaves this call omits are never read either, so an adversary holding both keys adds them and the receiver admits.

The presentation window intersects 4 sources: the receiver-held treaty, continuation, lease activation, and governance activation. The hook checks dispatch time against the intersection and the statement’s lease expiry against the issuer’s window. Lease issuer and scope, and governance issuer and receipt digest, must match locally provisioned records. A shorter lease expiry is permitted; an expired statement is not. Missing records and durable revocation tombstones deny. The same resolution runs again immediately before dispatch, without reusing an earlier validity decision. These are observations at admission and revalidation, not an atomic transaction with a concurrently changing external revocation authority. Store-reopen cases exercise missing, expired, future, revoked, and empty-window records through the dispatch counter. Provisioning remains trusted: a stored governance reference records what the receiver activated, and does not itself prove

the referenced receipt’s authenticity.

Revocation. Revocation is the only part of this protocol that depends on arrival. A kernel installs an origin’s signed snapshot only if its epoch strictly exceeds the installed one, by compare-and-swap, so neither a replay nor a lost concurrent install is possible, then applies its own freshness window to the issue time. Safety holds in every partition because both halves are local predicates that never wait for a message: no call is allowed against an observed revocation, and one evaluated against a stale snapshot is denied.

Liveness holds only under ASSUME-GOSSIP-FAIRNESS-PARTITION-BOUND, and bounds the time before a revocation is observed, not the number of evaluations, which is unbounded. With F the freshness window, T the control-poll period, L one admitted call’s latency, and k the calls made before one is decided against the newer snapshot, a still-connected receiver denies within $T + kL$ of publication and a cut receiver within $F + T + kL$, its snapshot first ageing past F . At $F = 500$ ms, $T = 250$ ms and the median L and k of Section 8, those are 663.6 ms connected and 1163.6 ms cut.

7 What is mechanized

The decision core is a pure function: verify a capability, resolve grants, evaluate, sign. It is modeled in Lean 4 [9], whose theorem inventory carries 196 declarations and one explicit axiom, a collision-resistance idealization for ASSUME-SHA256. No serializer, protocol step, or kernel behavior is axiomatized; an unproved property is an assumption-registry entry.

Two results bear on this paper. Occurrence theorems make the predicate language fail closed: an atom unsupported, or undefined for the admission view, anywhere in a predicate forces denial on every view, by induction. The second is the comparison set of Section 6: the classification of every leaf of the co-signed predicate is total by exhaustiveness over the leaf type, and acceptance holds exactly when the statement gate and those comparisons do.

The classification is cross-checked, not transcribed: the executed corpus derives its enumeration from the wire type and fails unless the Lean module agrees leaf for leaf on the wire path and the comparison kind. What each leaf is compared against is prose on both sides, checked on neither, so a classification naming the wrong quantity would pass.

Lean does not verify the running Rust. These theorems are about the modelled predicate and classification; that the shipped hook realizes them is a corpus over one fixture, not a theorem. Four tripwires fail loudly when the two drift apart: named Rust functions mirrored to Lean definitions by content hash; 20 production functions extracted through a Rust-to-Lean pipeline with equivalence theorems that must rebuild byte-identically; 47 bounded model-checking harnesses over public entry points; and the cross-language encoder corpus of Section 8.

We report survivors, not percentages. Of 45 Lean mutants 33 were killed and 12 survived, each with an open issue; of 166 in the Rust proof-facing sources 160 were killed, 5 were unviable, and 1 survived. That campaign predates the classification’s entry in the mutation allowlist, so it bears on the occurrence theorems, and the largest group of its survivors sits on the predicate language they are about.

8 Evaluation

Local measurements ran on one machine: 12 Neoverse-N1 cores, 46.9 GiB, Linux 6.17.0-1020-oracle aarch64, compiler 1.94.1 at the pinned toolchain. Percentiles interpolate between order statistics; each interval below is a percentile bootstrap interval for its median. These timings come from retained runs predating the new lease and governance activation checks; they are not latency measurements of the revised hook.

Two kernels in separate processes. Two kernels ran as separate operating-system processes, each with its own keys, stores, agreements, and receipt log, a third process driving calls over iroh QUIC, relays disabled; the topology record reports 1 hosts. The co-signing and revocation hops are the shipped lanes unchanged; the request hop uses a transport defined for this experiment, since no shipped lane carries a tool call. Admitted calls: 100 calls, p50 206.803 ms, interval 205.924 to 209.998 ms, p99 234.520 ms at the driver, each costing 3 co-signing connections, counted by the receiver. The round trip is the receiver’s decision plus the co-signing hops it makes back to the origin after dispatching, inside its 203.787 ms median evaluation window; minting the per-call evidence costs 3.297 ms more, outside it.

Denied calls: 7 scenarios over the wire, 140 calls, p50 58.821 ms, p99 88.462 ms. They return `chio_treaty_missing_scope`, `chio_treaty_scope_hash_mismatch`, `chio_treaty_missing_intersection`, `chio_`

Table 4: Local cost decomposition. The first two rows are complete kernel paths measured per invocation with both kernels in one process; the rest are components.

Path or component	p50	p99 or max	samples
Admitted call, both kernels in one process	13.863 ms	26.176 ms	200
Refused before dispatch	7.073 ms	18.646 ms	400
Receipt append to the store	4681.643 us	13566.644 us	500
Receipt signing	229.340 us	270.205 us	100
Receipt verification	337.470 us	344.582 us	100
Inbound two-signature statement verification	339.127 us	350.904 us	100
Cross-boundary admission decision	20.688 us	21.207 us	100

`treaty_intersection_mismatch`, `chio_treaty_missing_required_evidence`, `request_smuggled_trust_root`, and `request_smuggled_dynamic_trust`. In every one the receiver’s dispatch counter stayed zero. 88 calls were retried and excluded, denied on a snapshot aged past the 500 ms freshness window against an origin publishing every 250 ms.

Revoking at the origin and measuring to the receiver’s first denial took 322.899 ms at the median over 10 repeats, interval 178.977 to 368.288 ms, a median of 2 calls. With the link cut the receiver serves until its snapshot leaves the freshness window and then denies, 744.681 ms at the median over 10 repeats, interval 609.423 to 916.110 ms, a median of 2 calls, counting a denial only once it persists for three further calls. On 3 of 4 revoke records and 0 of 10 cut records the denial also reports an unconfirmed pre-dispatch cleanup: the budget unwind rather than the continuation release Section 4 orders, reporting a budget hold it cannot find, an invariant violation we do not explain. No call dispatched.

What admission costs locally. The shape of Table 4 is the result: the cryptography is not the expense. Appending the receipt durably costs 13.8 times what verifying the inbound statement costs and 226 times the receiver’s decision over already resolved evidence, where the comparisons of Table 3 run. Durability dominates, the price of Section 4’s ordering. The refusal row is a lower bound: its fixture refuses on a unanimous deny verdict, an early exit reaching neither those comparisons nor the signature checks. The retained sustained run admitted 3543 calls in 60 s at 59.0 per second, with 0 denials and 40.1 MiB of receipt-store growth. Its artifact does not record offered concurrency or a completed sweep, so these numbers do not establish saturation or maximum capacity. The newer concurrent driver requires a fresh retained run.

The negative corpus is 20 in-process cases, each asserting a refused decision and its code: 8 through the dispatch-capable hook and 12 against the verifier, alongside 20 bilateral and 50 generic replay cases.

What a composition of existing parts reaches. Table 5 prices what an operator adds; Table 6 says the explicit fields this profile omits. Hardening closes 9 attacks and carries 3 of the four properties, and 4 attacks survive it. The field ledger describes this profile, whose authorization server is controlled by the caller. It does not bound OAuth, JWT, or A2A: OAuth permits a receiver-owned issuer and opaque token lookup, and JWT permits application claims. A regression using the unchanged receiver and token types installs a separate receiver-owned issuer. Receiver issuance admits once; replay under a new message identifier and a caller-signed replacement both deny. This counterexample establishes issuer-placement flexibility, not complete binding or crash-recovery parity for the treaty protocol.

An independent opaque-handle ledger and Chio’s experimental effect slots receive identical evidence, owner rules, verifiers, SQLite durability and effect sinks. They match in sixteen cases, six state transitions, eight-process races and four kill boundaries per gate. Both publish once and reject replacement agents; uncertain post-claim effects remain unknown. Both also publish a checked Git-hook repair. An offline auditor rejects signed buggy source and accepts unsigned repairs without proving publication history. A three-receiver archival graph matches in eleven scenarios and six attacks, with twenty process kills. Namespaces isolate roles; the host and operator remain trusted. No integration-cost advantage is established.

A separate work profile connects a Python buyer, Rust provider and isolated Python procurement worker to a Rust specialist over HTTPS. Four distinct keys separate agents from kernel issuers. Buyer-approved authentication declarations alone cross to the specialist. Its kernel requires a provider-signed permit for the exact child job; a delegated key alone cannot buy another job. Both verifiers check nested delivery evidence. After a parent crash, agreed release can restore the buyer’s credit while the specialist remains paid. These local-credit observations establish neither atomic settlement across companies nor independently operated

Table 5: A receiver composed from 4 formats this project did not author, driven under two wirings against the threat table of this paper, case by case where the composition has an analogue. One call is a Model Context Protocol `tools/call` [2] inside an A2A message [1], presented over a channel authenticated by the peer’s SPIFFE X509-SVID [22] and carrying the access token of an RFC 8693 exchange [15], a JWT [14] in compact JWS [13] whose scope takes its syntax from OAuth 2.0 [11]. 9 published documents define every one of the 44 fields on the wire, and a test walks every byte an admissible call serializes and fails on any path none of them defines. *Composed* wires each part the way its own document does; *hardened* adds the deployment checks implemented in this artifact, among them a denylist over the context names the receiver owns, a second replay table over the credential’s own identifier, a bound on the window the issuer minted, and a comparison of the acting party against the workload the channel authenticated. It is the wiring we wrote, not a bound on the wiring an operator could write. Admissions are over the 23 attacks the corpus drives, as 23 distinct experiments. Chio’s own record is at most 15.3 times heavier per call, a bound rather than a measurement, since the two stores are read by two procedures, and its side is read from the run above rather than re-measured here.

Property of Section 6	Composed	Hardened
Admission binding	fails	fails
Receiver locality	fails	holds
Single use	holds, weakened	holds
Audience binding	holds	holds
Attacks admitted	13	4
Durable bytes per call	776.2	876.5

Table 6: Explicit carriers in the selected caller-issued profile. These counts describe its schema and configured checks, not all encodings the cited standards permit. The metadata witnesses show that unread metadata has no effect and that a receiver can compare a supplied version with its own record. Supplying the matching version is not by itself an authorization attack.

Facts	Where a value travels	Attested by	What making it resolvable takes
4	a scope value, the tool name beside it, an argument of the tool, and sub with act.sub over SPIFFE identifiers	the caller’s authorization server, or the caller	nothing: the receiver already resolves them
3	a task identifier and a reference list in the message body; the token’s exp and nbf	the caller, in bytes no signature in the set covers; the issuer, against a clock	a receiver-owned authorization record and an authenticated handle; OAuth permits this deployment
8	no explicit field in this selected profile	nobody	application claims or a receiver-owned lookup profile

deployment.

Verification by someone who was not there. The 50.6 KiB evidence package for the refund workflow verifies every receipt, statement, lease, and governance record in it with no kernel access, in 49.7ms at the median over 100 runs. A frozen corpus of 14 receipt cases holds four independent implementations to the same canonical body bytes and verdict.

Reproduction. The retained benchmark runs include per-host commit and environment records, raw samples and generated numeric macros checked against those artifacts. The historical composition benchmark records timings only with a clean input tree and an unloaded host. The newer outcome-gate experiment retains source hashes, input artifacts, receiver snapshots and test logs from an uncommitted local tree; it makes no timing claim.

9 Limitations and related work

The cross-organization run of Section 8 put the two kernels in separate processes on one host, not on two separately administered machines. That removes the shared address space, store, and key material, which are what the claim is about; the shared operating system and clock source remain. The rule does not depend on that experiment; the strength of the demonstration does.

Section 8’s alternative is one construction of those formats on one corpus: what it measures is what an operator must add, not a bound on what the parts reach. Continuation consumption is at-most-once, so a

crash between consume and dispatch loses the call rather than repeating it: right for a refund, an avoidable retry for an idempotent read. The chain closes in the kernel for one hop: the next continuation in a longer chain is minted outside it. Denial paths below the verifier still return a rendered detail alongside the code. Receipts batch into Merkle checkpoints in the RFC 6962 shape [17] with offline inclusion proofs, and that is the whole claim: local audit evidence with verifiable inclusion, not a public log, and with no anti-equivocation a kernel signing two contradictory checkpoints is detectable only by someone holding both.

Related work. Receiver-rooted policy and attenuated delegation are established by trust management [10, 6], proof-carrying authorization [3], end-to-end authorization [12], and macaroons [4]. Modern capability formats [24, 8] also support application-defined restrictions. These systems are relevant alternatives; their different wire shapes do not imply that they cannot express a Chio admission policy. Authenticated Workflows [18] also uses signed completion attestations to enforce workflow dependencies. Chaining outcome evidence alone is therefore not our novelty claim.

CapLease [25] studies repeated agent actions under freshly issued tokens. Its equally stateful server-ledger baseline obtains the same replay protection under matched centralized assumptions. Durable authorization state and an idempotent effect sink therefore belong in our comparison, not only token identifiers. Signatures and DSSE [21, 23] establish authorship and binding, while transparency systems [17, 5] address publication and equivocation under additional assumptions.

The result still needed. Local re-execution checks a source repair without its producer’s attestation, but cannot prove private publication history. The matched ledger reproduces these guarantees. Neither artifact validity nor signatures establish scarce work or Sybil resistance. Contingent payment additionally requires fair exchange [7]; a hash lock does not verify a hidden repair. Useful execution across independently administered receivers, with fewer recovery interventions or less application-specific security code than an equally provisioned alternative, remains unmeasured. Finite checking does not establish arbitrary correctness. Signatures do not make dishonest receivers’ external-effect claims true.

References

- [1] Agent2Agent Project. A2A: Agent2agent protocol specification. Specification, 2025.
- [2] Anthropic. Model context protocol specification. Specification, 2024.
- [3] Lujo Bauer, Michael A. Schneider, and Edward W. Felten. A general and flexible access-control system for the web. In *USENIX Security Symposium*, 2002.
- [4] Arnar Birgisson et al. Macaroons: Cookies with contextual caveats for decentralized authorization in the cloud. In *Network and Distributed System Security Symposium (NDSS)*, 2014.
- [5] Henk Birkholz et al. An architecture for trustworthy and transparent digital supply chains. IETF SCITT working group, Internet-Draft, 2025.
- [6] Matt Blaze et al. The KeyNote trust-management system version 2. RFC 2704, 1999.
- [7] Matteo Campanelli, Rosario Gennaro, Steven Goldfeder, and Luca Nizzardo. Zero-knowledge contingent payments revisited: Attacks and payments for services. *Cryptology ePrint Archive*, Paper 2017/566, 2017.
- [8] Geoffroy Couprie and Clément Delafargue. Biscuit: A decentralized authorization token. Specification, v3, 2023.
- [9] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *International Conference on Automated Deduction (CADE)*, 2021.
- [10] Carl M. Ellison et al. SPKI certificate theory. RFC 2693, 1999.
- [11] Dick Hardt. The OAuth 2.0 authorization framework. RFC 6749, 2012.
- [12] Jon Howell and David Kotz. End-to-end authorization. In *OSDI*, 2000.
- [13] Michael B. Jones et al. JSON web signature (JWS). RFC 7515, 2015.
- [14] Michael B. Jones et al. JSON web token (JWT). RFC 7519, 2015.
- [15] Michael B. Jones et al. OAuth 2.0 token exchange. RFC 8693, 2020.
- [16] Simon Josefsson and Ilari Liusvaara. Edwards-curve digital signature algorithm (EdDSA). RFC 8032, IETF, 2017.
- [17] Ben Laurie et al. Certificate transparency. RFC 6962, IETF, 2013.
- [18] Mohan Rajagopalan and Vinay Rao. Authenticated workflows: A systems approach to protecting agentic ai. arXiv:2602.10465v1, 2026.
- [19] Anders Rundgren et al. JSON canonicalization scheme (JCS). RFC 8785, IETF, 2020.
- [20] Jerome H. Saltzer et al. End-to-end arguments in system design. *ACM Transactions on Computer Systems*, 2(4):277–288, 1984.
- [21] Secure Systems Lab. DSSE: Dead simple signing envelope. Secure Systems Lab specification, v1.0, 2021.
- [22] SPIFFE Project. SPIFFE: Secure production identity framework for everyone. SPIFFE-ID, X509-SVID and JWT-SVID standards, 2024.
- [23] Santiago Torres-Arias et al. in-toto: Providing farm-to-table guarantees for bits and bytes. In *USENIX Security Symposium*, 2019.
- [24] UCAN Working Group. UCAN: User controlled authorization networks. Specification, v1.0, 2025.
- [25] Jinghan Xu, Longze Fan, Zeyuan Wang, Xinjin Li, and Hankai Liu. Beyond single-use tokens: Durable authorization state for replay-resistant llm agent actions. arXiv:2608.01710v1, 2026.